不同优先队列下 Dijkstra 算法的实现与评测

2026 年 10 月 28 日

摘要

实验在统一的 CSR 图表示和优先队列接口下，比较七种 Dijkstra 队列策略，包括 Fibonacci 堆，并另列 FIFO SPFA 与 SLF+LLL SPFA。数据包括随机图的 171 组配置、固定密度复杂度实验的 126 组配置，以及 NY、BAY 两个 DIMACS 路网各 1000 条查询、每图九种实现。WSL2 Release 实测中，路网上最快的 Dijkstra 队列是 radix heap，NY、BAY 的平均墙钟时间分别为 12.621 和 13.873 ms/查询；SLF+LLL SPFA 分别为 7.267 和 9.258 ms/查询。Fibonacci 堆没有在这两张路网上显示实际速度优势。报告分别讨论渐近保证、有限规模拟合和实测耗时，并说明整数键范围、输入可达性及重复次数的适用范围。

关键词：Dijkstra；Fibonacci 堆；radix heap；DIMACS；复杂度分析

目录

第 1 章 引言 **3**

1.1 问题与目标 3

1.2 实现与实验范围 3

1.3 实验说明 3

第 2 章 数据结构与算法说明 **3**

2.1 CSR 图与共享接口 3

2.2 Dijkstra 的主循环 4

2.3 二叉堆、段树与 PBDS 4

2.4 Fibonacci 堆 4

2.5 单调 radix heap 5

2.6 两种标签修正算法 5

第 3 章 实验结果 **6**

3.1 环境与计时说明 6

3.2 USA 路网输入与结果 6

3.3 随机图参数扫描 7

3.4 固定密度实验与拟合 8

第 4 章 分析与讨论 **10**

4.1 时间与空间复杂度 10

4.2 Fibonacci 堆的实际表现 11

4.3 Radix 和 SPFA 的差异 11

4.4 模型拟合与测量限制 12

4.5 后续工作 12

附录 A 源代码与复现说明 **13**

A.1 源码组织 13

A.2 随机图与固定密度图的可达性结果 14

参考文献 **15**

声明 **15**

第 1 章　引言

1.1　问题与目标

给定有向图 $G=(V,E)$、非负边权 $w:E\to\mathbb{Z}_{\ge 0}$ 和源点 s，求从 s 到各顶点的最短距离 $d[v]$。不可达顶点的距离记作 ∞。记 $n=|V|$、$m=|E|$；本文的 m 始终指有向弧条数。课程项目要求使用至少两种堆结构且包含 Fibonacci 堆，在 USA 路网中至少评测 1000 对查询，并给出运行时间、规模曲线与分析。

Dijkstra 的贪心顺序依赖最小优先队列。二叉堆、段树及 Fibonacci 堆对插入、减键和取最小元素的处理方式不同，因而具有不同的渐近代价和实际常数。距离为非负整数时，还可以使用单调的 radix heap。

我们探讨两个问题：减键在理论上更便宜，实际能否带来更快的搜索；以及固定图和输入之后，各实现的速度排序会怎样随图结构变化。

1.2　实现与实验范围

我们实现并比较了七种 Dijkstra 队列：标准库优先队列、手写二叉堆、段树堆、Fibonacci 堆、radix heap、GNU PBDS pairing heap 和 thin heap。除标准库版本以重复插入模拟减键外，其他版本支持按顶点定位并更新键。FIFO SPFA 与 SLF+LLL SPFA 是基于队列的标签修正算法，不属于 Dijkstra 的堆变体，在报告中单独列出。

所有实现共用 `general-dijkstra.hpp` 中的算法主循环以及 CSR 图。随机图与固定密度实验对同组参数使用固定种子 42 和同一个生成规则。路网采用 `USA-road-d.NY` 与 `USA-road-d.BAY`，查询文件是项目生成的 1000 对随机查询，种子为 971。

1.3　实验说明

路网输入形式虽然是点对点查询，现有算法仍从每个源点算完整个距离数组，直到队列为空，目标点不参与提前终止。所以本文的“ms/查询”是**一个点对触发一次完整单源搜索的平均时间**，不能等同于遇到目标点即停止的点对点搜索延迟。本轮不包含路径恢复、预处理索引或多线程并行搜索。

计时实验中，随机图和复杂度实验每组重复三轮；每个路网实现仅运行一个包含 1000 查询的批次。

第 2 章　数据结构与算法说明

2.1　CSR 图与共享接口

顶点编号为 $1,\dots,n$。CSR 使用连续边数组和长度为 $n+1$ 的偏移数组，按起点将弧分组；顺序访问某顶点的邻接弧需要与该顶点出度成正比的时间。边保存终点与整数权重，最短距离使用 64 位有符号整数。距离数组的初始无穷大取该类型最大值的四分之一，为本轮输入的距离加法留出余量；该实现没有覆盖任意大输入的通用溢出检测。

优先队列对外提供 `push_or_decrease(key,v)`、`pop()`、`empty()` 和 `clear()`。支持减键的实现通过顶点索引定位元素，共享基类用时间戳区分本轮在堆中的顶点。`pop()` 后清除对应在堆标记，完整搜索结束时队列必须为空并可复用。

2.2 Dijkstra 的主循环

初始时源点距离为零；每次松弛只用已发现路径的长度更新标签。标准库版本可能含有旧键，故主循环会跳过过期条目。

```
DIJKSTRA(G, s, Q):
  fill d[0..n] with INF
  d[s] = 0; Q.push_or_decrease(0, s)
  while not Q.empty():
    (key, u) = Q.pop()
    if key > d[u]: continue
    for each (v, w) in G[u]:
      if d[u] + w < d[v]:
        d[v] = d[u] + w
        Q.push_or_decrease(d[v], v)
  return d
```

代码清单 1: 共享 Dijkstra 伪代码

2.3 二叉堆、段树与 PBDS

标准库优先队列。最小堆保存 (d, v)。每次改进距离直接插入新条目，取出时丢弃旧条目。因此堆中可能同时有同一顶点的多个键，空间随成功松弛次数增长。

手写二叉堆。数组 `heap[pos]` 保存顶点，`idx[v]` 保存位置，`key[v]` 保存当前键。插入把顶点放到末尾并上浮；减键从 `idx[v]` 上浮；取最小值把末尾移到根并下沉，每次交换同步更新两个顶点的位置。每个在堆顶点只有一个条目。

段树堆。每个顶点对应一个叶子，内部结点保存子区间的最小键及其顶点。插入或减键沿根路径更新；取最小值读取根的顶点，再将相应叶子置为 ∞。更新过程中即使最小值不变，只要对应顶点改变也必须继续传播，以避免相等键导致过期顶点；最大编号顶点与哨兵叶子必须分开。

PBDS pairing 与 thin heap。适配器保存每个顶点的 `point_iterator`，首次入堆调用 `push`，随后用 `modify` 更新条目。内部堆序维护交给 GNU 库。本项目没有重新实现两种库堆，也不把它们的减键摊还界简单等同于 Fibonacci 堆。

2.4 Fibonacci 堆

每个顶点在预先分配的节点数组中对应一个槽位。节点包含键、父亲、孩子、左右兄弟指针、度数和标记；根表与孩子表均采用循环双向链表，另存最小根指针。插入只将单节点树并入根表；取最小值时提升其孩子，再按度数合并同度树。实现通过摘下当前根再处理的方式遍历根表，避免一边合并一边依赖失效的循环终止点。

```
INSERT(v, key):
  initialize node[v] as a singleton unmarked root
  splice node[v] into root list; update minimum
EXTRACT_MIN():
```

```
  remove minimum root z
  promote all children of z to unmarked roots
  while two roots have the same degree:
    link the larger-key root under the smaller-key root
  rebuild root list and minimum; return z
DECREASE(v, key):
  node[v].key = key
  if v violates heap order with its parent:
    cut v to the root list
    mark its parent on the first child loss
    repeatedly cut marked ancestors after another child loss
  update minimum if needed
```

代码清单 2: Fibonacci 堆的关键操作

标记记录非根节点是否已失去一个孩子。势函数可取 $\Phi = t + 2q$，其中 t 为根数，q 为标记节点数；级联剪切造成的标记减少可支付剪切代价。Fibonacci 堆由此支持摊还 $O(1)$ 的插入和减键、摊还 $O(\log n)$ 的取最小值，得到 Dijkstra 的 $O(m + n \log n)$ 界 [1]。该界描述总操作序列，并不保证每一次取最小值都只需对数时间。

2.5 单调 radix heap

radix heap 保存上次取出的键 L。键 k 放入编号 $b(k) = \mathrm{bitwidth}(k \operatorname{xor} L)$ 的桶，其中桶零只含等于 L 的键。若桶零为空，从最小非空桶找出最小键作为新 L，并按新 L 重新分配该桶；重新分配后的桶编号会减小。减键使用记录了桶编号和桶内位置的句柄，跨桶移动时交换删除并修复被交换元素的句柄。

```
POP():
  if bucket[0] is empty:
    b = smallest nonempty bucket id
    L = minimum key in bucket[b]
    redistribute every entry in bucket[b] using bitwidth(key XOR L)
  remove and return an entry from bucket[0]
  if all buckets are empty: reset L = 0 for the next query
```

代码清单 3: Radix heap 的取最小值

合法插入要求 $k \geq L$，而非要求所有插入键按全局顺序递增。Dijkstra 的非负边权保证新发现候选距离不小于当前取出的有效距离，满足这一约束。当前实现只有 32 个桶，桶内位置句柄占 25 位；它不是任意 64 位键范围上的通用 radix heap。

2.6 两种标签修正算法

FIFO SPFA。队列存 (d, v) 条目；成功松弛后插入新条目，取出后跳过过期距离。它没有单独的在队标记，可能出现重复条目，因此内存和操作次数不应直接套用“一顶点一个队列项”的分析。

SLF+LLL SPFA。双端队列只存顶点，并维护在队标记。成功松弛未在队的顶点时，若其标签小于队首则从前端插入（SLF），否则插到后端。处理队首前，若其标签大于在队顶点的平均标签，则移至队尾（LLL）。通过标签总和维护平均值，队内顶点标签减少时同步更新总和。两条启发规则只调整松弛顺序，不提供 Dijkstra 式的出队定型保证。

```
ENQUEUE(v):
  if queue empty or d[v] < d[front]: push_front(v)
  else: push_back(v)
  in_queue[v] = true; sum += d[v]; count += 1
PROCESS():
  while d[front] * count > sum: rotate front to back
  u = pop_front(); in_queue[u] = false
  sum -= d[u]; count -= 1
  relax outgoing arcs of u
  update sum on an already queued label decrease
  otherwise enqueue each newly improved vertex
```

代码清单 4: SLF+LLL 的队列调度规则

第 3 章 实验结果

3.1 环境与计时说明

表 1: 实验环境与配置

项目	配置
平台	同一台 Intel Core i9-14900HX；WSL2 Arch Linux，32 个逻辑 CPU
工具链	GCC 16.2.1；CMake 4.4.3；Ninja 1.13.2；Google Benchmark 1.9.5
编译	C++20，Release，`-O3 -DNDEBUG`
输入与执行	二进制与图数据位于 WSL ext4
运行策略	benchmark 串行执行；随机图和复杂度各三轮；路网每实现一个完整批次
计时	自动迭代最短目标 0.5 s；路网每次迭代固定包含 1000 查询

图生成、文本解析、优先队列构造与距离数组初次分配位于计时循环外；每次搜索对距离数组的重新初始化仍在计时范围内。SLF+LLL 的局部在队标记数组分配在其搜索函数中，包含在计时中。随机图与复杂度曲线使用每次调用的 CPU 时间，误差棒为三轮结果的**样本标准差**，不是置信区间，也不是 1000 查询的离散程度。路网表使用搜索循环中墙钟时间除以查询数；原始 CSV 同时保留整批 CPU 与墙钟数据。

3.2 USA 路网输入与结果

NY 图有 $n = 264346$、$m = 733846$，BAY 图有 $n = 321270$、$m = 800172$；两图均使用完整的 1000 对查询。每个实现运行一次完整批次，搜索总计 $9 \times 2 \times 1000$ 次。路网数据来自第 9 届 DIMACS 挑战赛 [3]。

表 2: 平均每查询墙钟耗时；加速比为标准库耗时除以当前实现耗时

实现	NY/ms	BAY/ms	NY/加速比	BAY/加速比
std PQ	19.849	20.518	1.00	1.00
Binary heap	19.720	20.630	1.01	0.99
Segment tree	36.344	42.261	0.55	0.49
Fibonacci heap	44.803	51.184	0.44	0.40
Radix heap	12.621	13.873	1.57	1.48
PBDS pairing	26.985	30.088	0.74	0.68
PBDS thin	36.818	42.577	0.54	0.48
FIFO SPFA	249.409	353.306	0.08	0.06
SLF+LLL SPFA	7.267	9.258	2.73	2.22

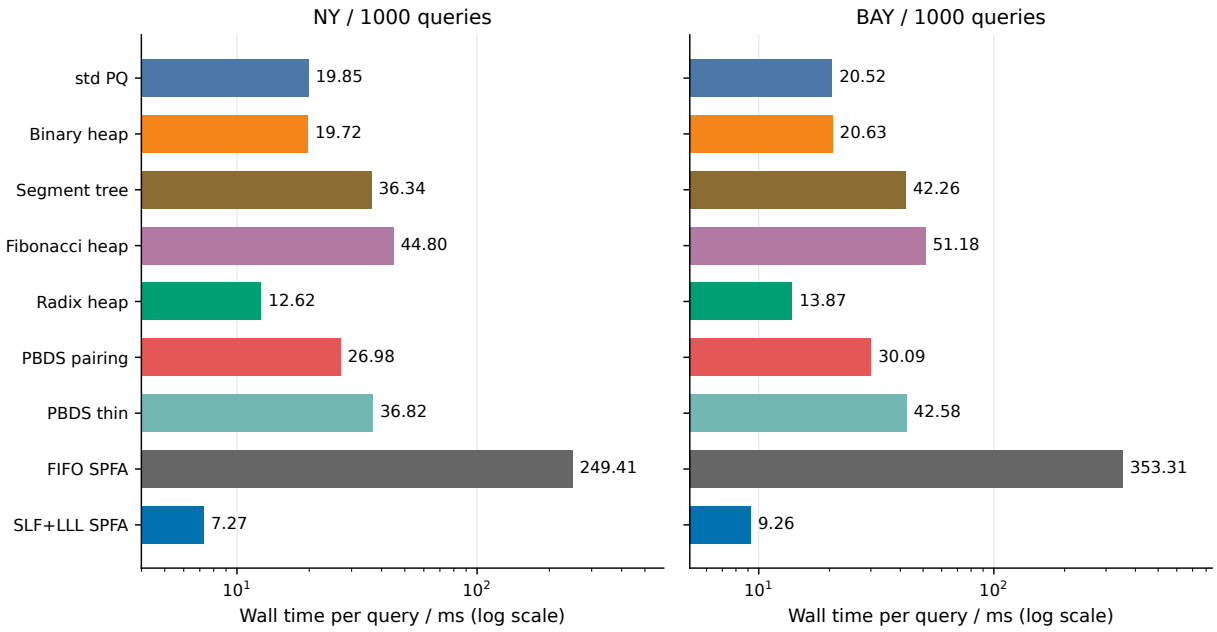


图 1: NY 与 BAY 的平均查询耗时。横轴为对数；每项均为一个完整 1000 查询批次，未估计重复运行方差。

NY/BAY 中 radix 相对标准库的加速比分别约为 1.57、1.48，SLF+LLL 为 2.73、2.22；Fibonacci 的耗时则为标准库的 2.26、2.49 倍。FIFO SPFA 分别需要 249.409、353.306 ms/查询，明显更慢。这些结果只覆盖本轮输入，不能推出 FIFO 在所有图上都慢，也不能说明 SLF+LLL 总有优势。

3.3 随机图参数扫描

随机有向图共有 19 组 (n, m)、每组九种实现，合计 171 个配置。允许重边，排除自环，权重范围为 $[0, 200000]$，源点为 1，图不保证连通。同一 C++ 生成器记录的可达性显示：$n = 163840$ 的两组图（$m = 163839, 327678$）及 $n = 10, m = 9$ 都仅源点自身可达，可达弧数为零。此时计时

主要反映初始化和调用开销，不能代表遍历全图的代价。完整可达性结果见附录代码清单 6，原始计时保持不变。

表 3: 三组随机图的平均 CPU 时间，单位 ms；对应弧数依次为 327552、327672、327676

实现	$n = 2560$	$n = 40960$	$n = 81920$
std PQ	1.111	8.050	11.845
Binary heap	0.518	6.046	11.771
Segment tree	0.575	6.468	12.769
Fibonacci heap	0.808	12.751	26.152
Radix heap	0.424	3.246	6.252
PBDS pairing	0.736	9.412	18.288
PBDS thin	1.008	12.700	23.544
FIFO SPFA	0.646	3.710	5.074
SLF+LLL SPFA	0.555	2.960	3.751

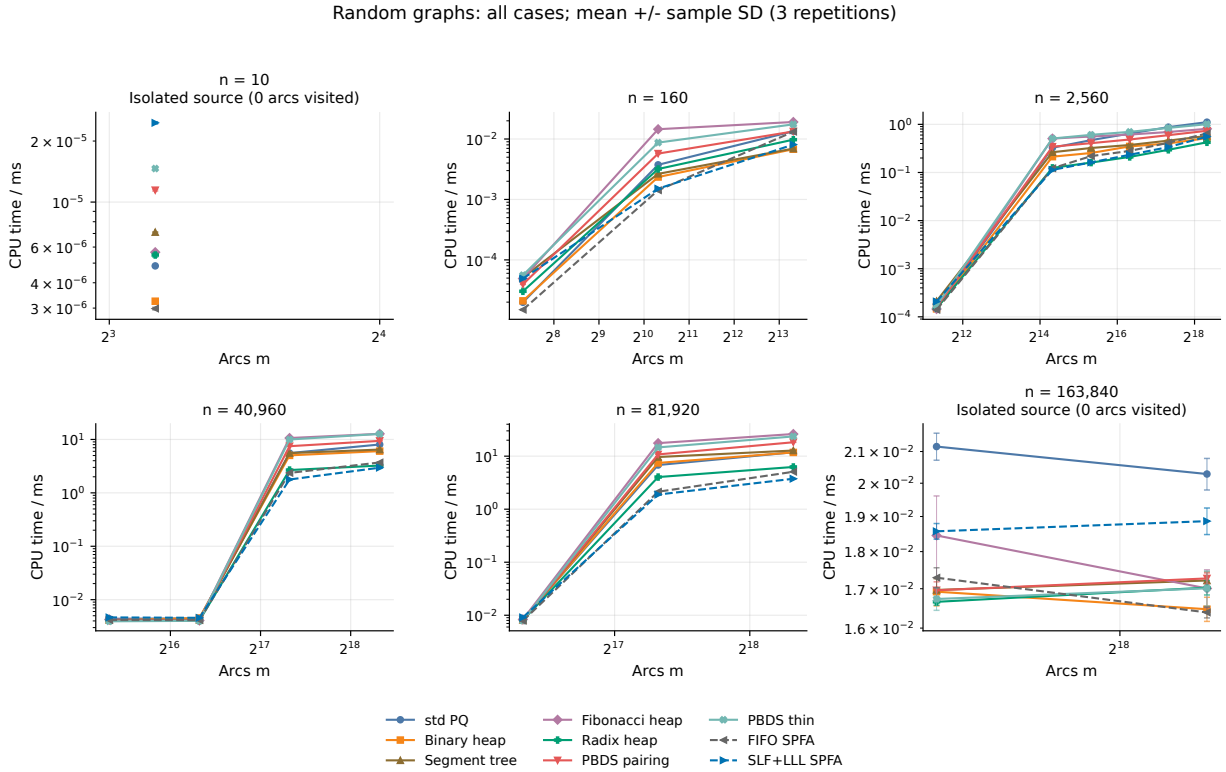


图 2: 随机图全部参数，按固定顶点数分面，只在同一面板内改变弧数。误差棒为三轮样本标准差，纵轴为对数；孤立源点已在面板标题标注。

3.4 固定密度实验与拟合

固定密度实验使用两族图：稀疏族 $m = 4n$，$n = 2^{10}, \ldots, 2^{17}$；稠密族 $m = n(n-1)/4$，$n = 2^{7}, \ldots, 2^{12}$。每族每个规模都测量九种实现，合计 $9(8+6) = 126$ 组。

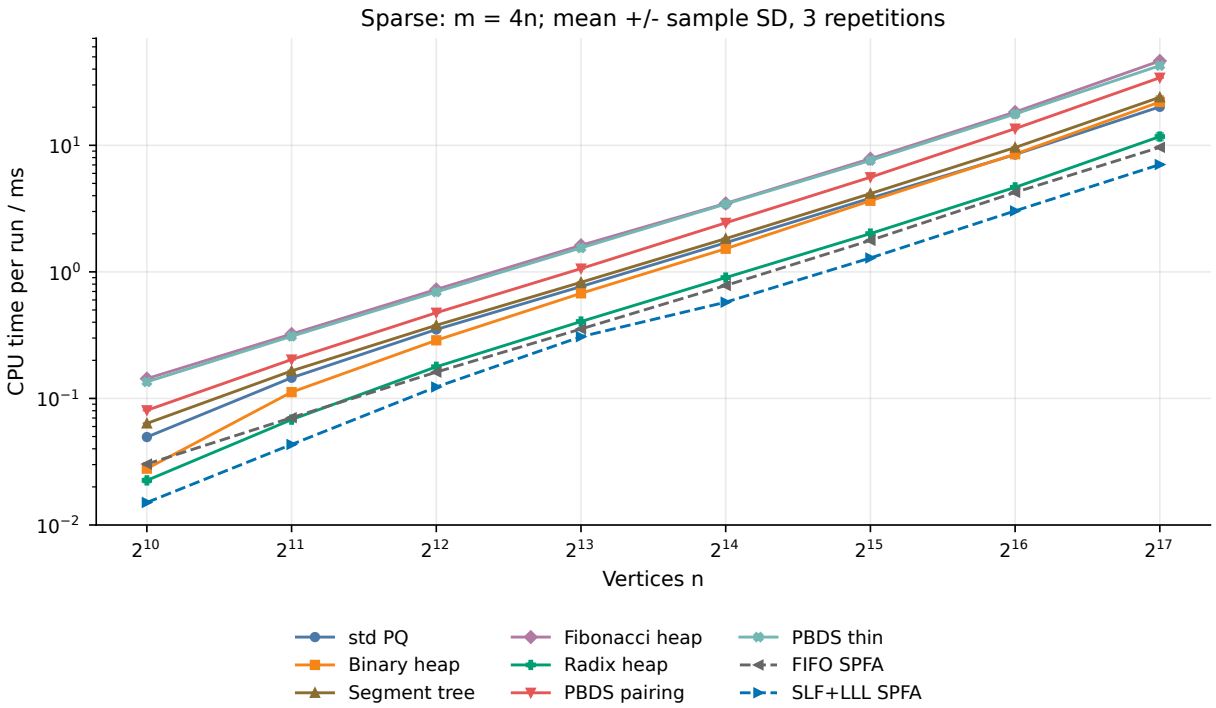


图 3: 稀疏族的平均 CPU 时间，$m = 4n$。误差棒表示三轮样本标准差。

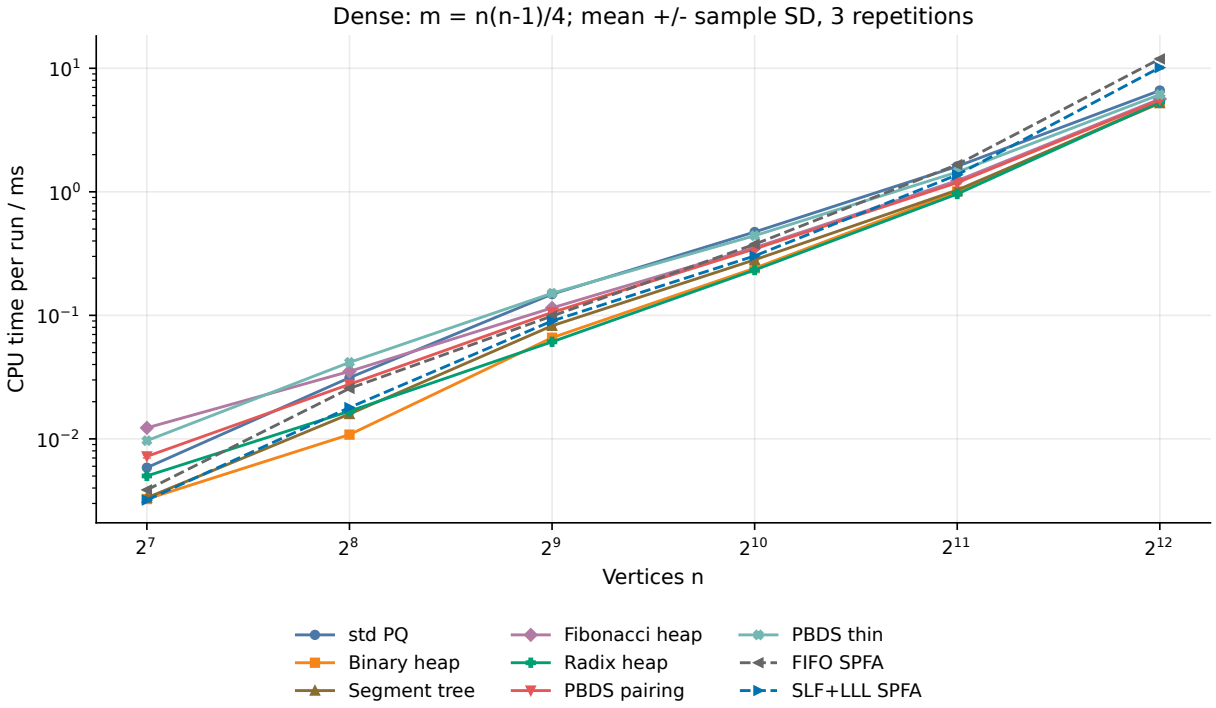


图 4: 稠密族的平均 CPU 时间，$m = n(n-1)/4$。误差棒表示三轮样本标准差。

Google Benchmark 分别固定 $f(n) = n \log_2 n$ 和 $f(n) = n^2$，用过原点的最小二乘形式 $T(n) \approx cf(n)$ 拟合。表4 的稀疏系数单位为 $\mathrm{ns}/(n \log_2 n)$，稠密系数为 ns/n^2，两列系数不能直接互相比较大小。RMS 为库定义的归一化残差百分比，衡量有限范围内模型贴合程度。

表 4: 固定模型的 CPU 系数与 RMS；所有值由 CSV 自动导出

实现	稀疏系数	稀疏 RMS	稠密系数	稠密 RMS
std PQ	8.806	9.1%	0.393	8.0%
Binary heap	9.444	16.4%	0.314	14.3%
Segment tree	10.361	13.4%	0.307	10.7%
Fibonacci heap	20.008	14.5%	0.334	5.9%
Radix heap	5.063	16.4%	0.313	14.1%
PBDS pairing	14.729	15.1%	0.327	6.8%
PBDS thin	18.562	10.9%	0.364	3.9%
FIFO SPFA	4.245	8.1%	0.690	23.2%
SLF+LLL SPFA	3.089	8.7%	0.586	24.3%

同样的模型用于 SPFA 只作为经验拟合，不用于推导其渐近界。稠密图下 Fibonacci 的理论项为 $m + n \log n = \Theta(n^2)$；堆 Dijkstra 的常规上界则含有 $m \log n$。有限规模内，扫描边的代价仍可能主导计时，因此 n^2 拟合得较好，并不表示对数因子已经被消掉。

第 4 章　分析与讨论

4.1　时间与空间复杂度

表5 讨论从给定源点进行一次完整搜索。图本身占 $O(n + m)$ 空间，距离数组占 $O(n)$；表中的队列空间单列。标准库版本使用懒删除，其堆中可能保留 $O(m)$ 个条目，与每个顶点仅一个在堆条目的减键实现存在实质差别。

表 5: 队列策略的代价与适用条件

策略	时间上界或分析说明	适用条件	队列空间
标准库懒删除	$O(n + m \log(m + 1))$	非负权；可重复条目	$O(m + 1)$
索引二叉堆	$O((n + m) \log(n + 1))$	非负权；可直接减键	$O(n)$
段树堆	$O((n + m) \log(n + 1))$	顶点固定映射为叶子	$O(n)$
Fibonacci 堆	$O(m + n \log(n + 1))$，摊还	非负权；直接定位节点	$O(n)$
PBDS 两种堆	依库堆的具体摊还界，不在本项目中推导	GCC 扩展与有效句柄	$O(n)$
Radix heap	按键位宽 B 计为 $O(m + nB)$ 的桶迁移分析	非负整数；单调键与容量限制	$O(n + B)$
FIFO SPFA	与成功松弛和重复入队有关，存在坏例	本实验只使用非负权	依实际条目数
SLF+LLL	启发式不能保证 Dijkstra 式效率	本实验只使用非负权	$O(n)$

对 Fibonacci 堆，所有弧扫描和成功减键总计 $O(m)$，每个顶点至多一次有效取最小值，合计

$O(n \log n)$。稀疏图上该界与二叉堆同阶，但 Fibonacci 堆每次操作要维护更多链接，理论差异不一定能换来更小的常数。Radix heap 的分析利用桶重分配时编号降低；若最大有效键界为 U，可令 $B = \lceil \log_2(U+1) \rceil$。文献中的专门 radix 设计可给出 $O(m + n \log C)$ 的 Dijkstra 界（C 为边权上界）[2]；本项目的有限 32 桶和压缩句柄实现与文献设定并不完全对应，因此不直接采用更紧的界。

本轮没有采集各算法的峰值内存曲线、硬件缓存计数、成功松弛次数或桶重分配次数。空间复杂度分析来自数据结构，不能当作已经完成的内存实测。

4.2 Fibonacci 堆的实际表现

本轮 NY、BAY 的平均出度只有约 2.78 和 2.49，属于稀疏路网。Fibonacci 堆减少了减键的渐近代价，却仍要合并同度根、维护链表并执行级联剪切。节点虽然一次性分配在数组中，每次操作仍沿节点链接跳转；二叉堆的大部分状态则是紧凑数组。更频繁的链接操作和访问局部性差异可以解释部分差距。这只是根据实现结构作出的推断，还需补上性能计数器和操作次数数据。本轮没有采集可直接区分缓存未命中的指标。

稠密族中边扫描工作随 n^2 增长，堆维护所占比重可能下降，某些队列之间的差距因此缩小。本轮最大稠密规模只有 $n = 4096$，还不能回答在足够大的规模上谁最终胜出。随机图的最大顶点规模虽较大，稀疏度却完全不同，不能代替大规模稠密实验。

4.3 Radix 和 SPFA 的差异

在非负整数路网上，radix heap 利用键的单调性，减少比较堆中的上浮和下沉工作，成为本轮七种 Dijkstra 队列中最快的一种。它仍要求正确的键范围与句柄容量，不能直接替换一般实数键或非单调任务中的优先队列。

SLF+LLL 在两图上比所有堆版本更快，FIFO 版本却慢一个数量级以上。同样是 SPFA，调度顺序换一下，结果就相差很大。两种实现还分别使用重复条目与在队标记，故差距也不能只归因于 SLF/LLL 一个因素。随机图上的良好表现不构成对抗输入下的效率保证。

4.4 模型拟合与测量限制

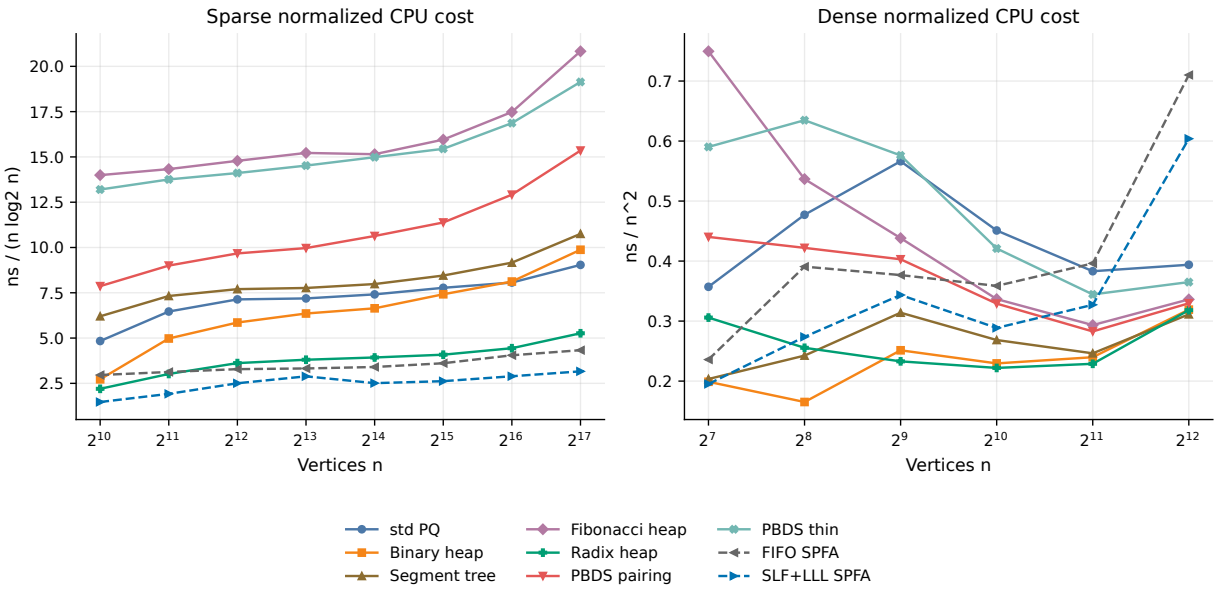


图 5: 按预设模型归一化的 CPU 时间。曲线并非严格常数；该图只用于观察模型偏差与有限规模效应。

复杂度图记录的是有限个离散规模上的耗时。归一化曲线的变化既可能来自算法操作数，也可能来自缓存、编译器、频率或输入可达性。RMS 小只说明所选范围内模型贴合，不能排除其他模型，也不能据此推出紧确界。

三轮随机/复杂度实验不足以稳定估计尾部分布；路网只有单批次、无逐查询分布，两个路网规模也不足以作可靠的规模拟合。WSL 共享宿主硬件，本轮未固定 CPU 亲和性或测量宿主负载，绝对时间只能作为本轮参考。

4.5 后续工作

后续应增加路网独立批次与逐查询记录，扩大图规模；记录松弛、过期条目、合并与桶迁移次数，并补充性能计数器数据来解释常数开销。若研究真正的点对点查询，应统一加入目标提前终止，再考虑双向搜索或预处理算法。

这轮数据里，摊还界并没有决定实际排序。稀疏路网上，Fibonacci 堆的理论优势没有转化成速度；radix heap 和调度合适的 SPFA 更快。这个排序仍只适用于当前输入和实现。

附录 A　源代码与复现说明

A.1　源码组织

源码位于项目的 pq-test/ 下。以下列出与本报告对应的主要文件。

表 6: 主要源码与功能对应

文件	功能
csr-graph.hpp、dimacs-parser.hpp	共享图结构与文件解析
general-dijkstra.hpp	共用 Dijkstra 主循环
heap-adaptor.hpp	时间戳接口、标准库与 PBDS 适配器
binary-heap.hpp、segment-tree-heap.hpp	二叉堆与段树
fibonacci-heap.hpp、radix-heap.hpp	Fibonacci 与 radix heap
general-bellman-ford.hpp	两种标签修正算法
dij-bench-v2.cpp	随机图参数扫描
dij-bench-complexity.cpp	固定密度复杂度实验
dij-bench-dimacs.cpp	路网查询批次基准

下列节选了 general-dijkstra.hpp 的主循环，展示了如何在不同堆实现间共享核心逻辑。

```cpp
// Shared Dijkstra loop, excerpted without identifying file-header comments.
// The caller allocates the distance vector and supplies a reusable empty heap.
template <typename Heap>
inline void dijkstra(const Graph &g, uint32_t s,
                     std::vector<int64_t> &dis, Heap &heap) {
  // Leave headroom for distance addition on the validated input domain.
  constexpr int64_t INF = std::numeric_limits<int64_t>::max() / 4;
  // Initialization is part of every timed search.
  std::fill(dis.begin(), dis.end(), INF);
  heap.push_or_decrease(0, s);
  dis[s] = 0;
  while (!heap.empty()) {
    auto [d, u] = heap.pop();
    // Standard-library lazy insertion can leave outdated entries.
    if (d > dis[u]) continue;
    // CSR adjacency traversal is shared by all heap variants.
    for (auto [v, w] : g[u]) {
      auto nd = dis[u] + w;
      if (dis[v] > nd) {
        dis[v] = nd;
        // Insert on first discovery, decrease an existing key otherwise.
        heap.push_or_decrease(dis[v], v);
      }
    }
  }
  // This implementation deliberately drains the heap, even for p2p input.
}
```

代码清单 5: 共享主循环节选

A.2 随机图与固定密度图的可达性结果

下表给出各规模图的顶点数与可达弧数，供解释随机图和复杂度实验的计时结果使用。

```
suite,scenario,n,m,reachable_vertices,reachable_arcs
dij-v2,random,10,9,1,0
dij-v2,random,160,159,3,2
dij-v2,random,160,1272,159,1260
dij-v2,random,160,10176,160,10176
dij-v2,random,2560,2559,3,2
dij-v2,random,2560,20472,2558,20456
dij-v2,random,2560,40944,2560,40944
dij-v2,random,2560,81888,2560,81888
dij-v2,random,2560,163776,2560,163776
dij-v2,random,2560,327552,2560,327552
dij-v2,random,40960,40959,2,1
dij-v2,random,40960,81918,2,1
dij-v2,random,40960,163836,40210,160865
dij-v2,random,40960,327672,40945,327535
dij-v2,random,81920,81919,1,0
dij-v2,random,81920,163838,65159,130698
dij-v2,random,81920,327676,80232,321087
dij-v2,random,163840,163839,1,0
dij-v2,random,163840,327678,1,0
complexity,sparse,1024,4096,999,3986
complexity,sparse,2048,8192,2015,8070
complexity,sparse,4096,16384,4007,16039
complexity,sparse,8192,32768,8047,32175
complexity,sparse,16384,65536,16057,64227
complexity,sparse,32768,131072,32124,128497
complexity,sparse,65536,262144,64217,256877
complexity,sparse,131072,524288,128488,513988
complexity,dense,128,4064,128,4064
complexity,dense,256,16320,256,16320
complexity,dense,512,65408,512,65408
complexity,dense,1024,261888,1024,261888
complexity,dense,2048,1048064,2048,1048064
complexity,dense,4096,4193280,4096,4193280
```

代码清单 6: 随机图与固定密度图的可达性结果

参考文献

[1] M. L. Fredman, R. E. Tarjan. “Fibonacci heaps and their uses in improved network optimization algorithms.” *Journal of the ACM*, 34(3), pp. 596–615, 1987. https://doi.org/10.1145/28869.28874.

[2] R. K. Ahuja, K. Mehlhorn, J. B. Orlin, R. E. Tarjan. “Faster algorithms for the shortest path problem.” *Journal of the ACM*, 37(2), pp. 213–223, 1990. https://doi.org/10.1145/77600.77615.

[3] C. Demetrescu, A. V. Goldberg, D. S. Johnson. *9th DIMACS Implementation Challenge: Shortest Paths*. 官方数据目录：http://www.diag.uniroma1.it/challenge9/data/USA-road-d/. NY/BAY 的原始文件名分别为 `USA-road-d.NY.gr.gz` 与 `USA-road-d.BAY.gr.gz`。本项目的数据来源与归档信息见 `docs/DATA.md`。

[4] Google. *Google Benchmark User Guide*, vendored version 1.9.5. https://github.com/google/benchmark/blob/v1.9.5/docs/user_guide.md. 本项目拟合及统计字段同时可在本地库实现中核查。

[5] Free Software Foundation. *Policy-Based Data Structures, libstdc++ Manual*. https://gcc.gnu.org/onlinedocs/libstdc++/manual/policy_data_structures.html.

声明

We hereby declare that all the work done in this project titled ”Shortest Path Algorithm with Heaps” is of our independent effort as a group.